

实 验 报 告

课程名称：线性回归

姓名：王志豪

学号：08232337

班级：2023 大数据一班

日期：2026. 3. 30

中国矿业大学计算机学院/人工智能学院

实验名称 线性回归 实验日期 2026. 3. 30

实验报告内容: 1. 实验目的 2. 实验内容 3. 实验步骤 4. 实验结果 5. 实验体会

一、实验目的

通过本实验掌握机器学习及优化课程的相关概念、技术、原理、应用等，提高编写实验报告、总结实验结果的能力，使对机器学习模型、算法等有比较深入的认识。所需掌握的知识点如下：

- 掌握机器学习中相关概念、模型、算法；
- 熟悉机器学习模型训练、验证、测试的流程；
- 熟悉常用的数据预处理方法；
- 掌握线性回归优化问题的表示、求解及编程。

二、实验内容

基于 California Housing Prices 数据集，完成关于房价预测的线性回归模型训练、测试与评估。

1. 准备数据集并认识数据

下载 California Housing Prices 数据集

<https://www.kaggle.com/camnugent/california-housing-prices>

了解数据集各个维度特征及预测值的含义

2. 探索数据并预处理数据

观察数据集各个维度特征及预测值的数值类型与分布

预处理各维度特征（如将类别型维度 `ocean_proximity` 转换为 `one-hot` 形式的数值数据）。

参考：<https://blog.csdn.net/SanyHo/article/details/105304292>

划分 70% 的样本作为训练数据集，30% 的样本作为测试数据集

3. 求解模型参数

编程实现线性回归模型的拟合形式参数求解

编程实现线性回归模型的梯度下降参数优化

4. 测试和评估模型

在测试数据集上计算所训练模型的 R^2 指标

三、实验步骤

实验原理：

(1) 介绍线性回归闭合形式参数求解的原理

线性回归的闭合形式参数求解旨在通过最小化损失函数（即预测值与真实值之间的均方误差），一步到位地求解出全局最优的模型参数。该方法无需像梯度下降那样进行多次迭代，直接通过求解偏导方程获得参数。

假设给定包含 N 个样本的数据集 $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$ ，其中每个样本特征为 d 维向量 $x_i = (x_{i1}, x_{i2}, \dots, x_{id})$ 。线性模型试图学习到参数向量 $\theta = (\theta_0, \theta_1, \dots, \theta_d)$ ，使得预测函数为：

$$f_{\theta}(x_i) = \theta_0 + \theta_1 x_{i1} + \dots + \theta_d x_{id} = x_i^T \theta$$

为了方便矩阵运算，我们构建样本特征矩阵 X 和目标值向量 y ：

$$X = \begin{bmatrix} 1 & x_{11} & \dots & x_{1d} \\ 1 & x_{21} & \dots & x_{2d} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_{N1} & \dots & x_{Nd} \end{bmatrix}, y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix}$$

整个数据集的预测向量可以表示为 $\hat{y} = X\theta$ 。由此，我们可以定义均方误差损失函数 J ：

$$J = \frac{1}{2N} \sum_{i=1}^N (f_{\theta}(x_i) - y_i)^2 = \frac{1}{2N} \|X\theta - y\|^2$$

将其展开为矩阵乘法形式：

$$\begin{aligned} J &= \frac{1}{2N} (X\theta - y)^T (X\theta - y) \\ J &= \frac{1}{2N} (\theta^T X^T X \theta - \theta^T X^T y - y^T X \theta + y^T y) \\ J &= \frac{1}{2N} (\theta^T X^T X \theta - 2\theta^T X^T y + y^T y) \end{aligned}$$

为了求极小值，我们对参数 θ 求梯度并令其为 0：

$$\nabla_{\theta} J = \frac{1}{N} (X^T X \theta - X^T y) = 0$$

化简得到正规方程（Normal Equation）：

$$X^T X \theta = X^T y$$

如果矩阵 $X^T X$ 可逆，即可直接解得最优参数 θ ：

$$\theta = (X^T X)^{-1} X^T y$$

(2) 介绍线性回归梯度下降参数求解的原理

当数据量极大或特征之间存在高度共线性导致矩阵不可逆时，闭式解不再适用。此时可采用梯度下降法（Gradient Descent）。它是一种迭代优化算法：从一个随机初始化的参数值出发，每次沿着损失函数下降最快的方向（即负梯度方向）更新参数，直到损失函数收敛至最优解。

损失函数与前文一致：

$$J = \frac{1}{2N} \sum_{i=1}^N (x_i^T \theta - y_i)^2$$

参数的迭代更新公式为（其中 η 为学习率，控制每次更新的步长）：

$$\theta = \theta - \eta \nabla J$$

梯度的详细推导过程如下：

$$\begin{aligned}\theta &= \theta - \eta \nabla \left(\frac{1}{2N} (X\theta - y)^T (X\theta - y) \right) \\ \theta &= \theta - \eta \left(\frac{1}{2N} \frac{\partial}{\partial \theta} (X\theta - y)^T (X\theta - y) \right) \\ \theta &= \theta - \eta \left(\frac{1}{2N} \cdot 2(X^T X\theta - X^T y) \right)\end{aligned}$$

最终得到梯度更新法则：

$$\theta = \theta - \eta \left(\frac{1}{N} X^T (X\theta - y) \right)$$

(3) 归一化

在多特征模型中，不同特征往往具有不同的量纲和取值范围（例如年龄在 1-100 之间，而身高在 0.4-2.5 之间）。如果在计算欧氏距离或进行梯度下降时不对特征进行处理，数值范围大的特征会主导模型，导致模型无法有效学习其他特征的信息。因此，需要统一各个特征的尺度。

归一化 (Min-Max Scaling): 将数据映射到特定的区间（通常是 [0, 1]）。

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)}$$

均值归一化 (Mean Normalization): 将数据中心化，使得均值为 0。

$$x' = \frac{x - \text{mean}(x)}{\max(x) - \text{mean}(x)}$$

标准化 (Standardization / Z-score): 将数据转化为标准正态分布（均值为 0，标准差为 1）。

$$x' = \frac{x - \text{mean}(x)}{\text{std}(x)}$$

(4) 求解 R²

R² 是评估回归模型拟合优度的核心指标。它反映了目标变量的波动中有多少比例可以被模型解释。**R²** 的取值范围通常在 0 到 1 之间，越接近 1 说明模型拟合效果越好。

SSE（残差平方和，Sum of Squares for Error）: 衡量模型预测值与真实值之间的误差。

$$SSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

SSR（回归平方和，Sum of Squares for Regression）: 衡量预测值与均值之间的差异，即模型能够解释的变异部分。

$$SSR = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - \bar{y})^2$$

SST（总离差平方和，Total Sum of Squares）: 衡量真实数据本身的总体波动程度。

$$SST = \frac{1}{N} \sum_{i=1}^N (y_i - \bar{y})^2$$

这三者之间存在如下等式关系：

$$SST = SSE + SSR$$

根据定义， R^2 等于回归平方和在总离差平方和中所占的比例：

$$R^2 = \frac{SSR}{SST} = 1 - \frac{SSE}{SST}$$

1. 数据集准备与探索

1.1 使用 pandas 框架加载 csv 文件的数据

```
code > dataset.py > ...
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 from sklearn.model_selection import train_test_split
5 data = pd.read_csv('housing.csv')
6 print(data)
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population	households	median_income	median_house_value	ocean_proximity
0	-122.23	37.88	41.0	880.0	129.0	322.0	126.0	8.3252	452600.0	NEAR BAY
1	-122.22	37.86	21.0	7099.0	1106.0	2401.0	1138.0	8.3014	358500.0	NEAR BAY
2	-122.24	37.85	52.0	1467.0	190.0	496.0	177.0	7.2574	352100.0	NEAR BAY
3	-122.25	37.85	52.0	1274.0	235.0	558.0	219.0	5.6431	341300.0	NEAR BAY
4	-122.25	37.85	52.0	1627.0	280.0	565.0	259.0	3.8462	342200.0	NEAR BAY
...	...	...	...	...	...	...	...	...	...	...
20635	-121.09	39.48	25.0	1665.0	374.0	845.0	330.0	1.5603	78100.0	INLAND
20636	-121.21	39.49	18.0	697.0	150.0	356.0	114.0	2.5568	77100.0	INLAND
20637	-121.22	39.43	17.0	2254.0	485.0	1007.0	433.0	1.7000	92300.0	INLAND
20638	-121.32	39.43	18.0	1860.0	409.0	741.0	349.0	1.8672	84700.0	INLAND
20639	-121.24	39.37	16.0	2785.0	616.0	1387.0	530.0	2.3886	89400.0	INLAND

可见，共有 20640 rows x 10 columns。纵列的特征分别为：经度、纬度、房屋的中位数年龄、房间总数、卧室总数、人口数、家庭数、收入中位数、房价中位数、海洋邻近。

各个特征的解释如下：

- **经度 (Longitude)**：房屋所在位置的经度，全部用负数表示，因为该位置位于西半球，加利福尼亚州的经度范围大致在 -114 到 -124 之间。经度数值越接近零，地理位置越向东。经度主要集中在 -122 附近，这些房屋位置在加州的西部海岸地区，靠近旧金山湾区等地区。
- **纬度 (Latitude)**：房屋所在位置的纬度，数值越大表示越北。
- **中位数年龄 (Housing Median Age)**：该位置房屋的中位数年龄，代表房屋的新旧程度。
- **房间总数 (Total Rooms)**：该区域所有房屋的房间总数。
- **卧室总数 (Total Bedrooms)**：该区域所有房屋的卧室总数。
- **人口数 (Population)**：该区域的总人口数量。
- **家庭数 (Households)**：该区域的家庭总数，通常是指居住单元数量。
- **收入中位数 (Median Income)**：该区域的收入中位数，单位是 1 万美元，反映区域的经济水平。
- **房价中位数 (Median House Value)**：目标变量，即房屋价格的中位数，单位为美元。
- **海洋邻近 (Ocean Proximity)**：类别变量，表示房屋距离海洋的距离。值包括“<1H OCEAN”（距海洋不到 1 小时）、INLAND（内陆）、NEAR OCEAN（靠近海洋）等。

1.2 数值类型与分布

- **Longitude (经度)**：连续型数值，范围大约为 -124 到 -114。左偏，值越大表示位置越东，集中在 -122 附近（加州西部）。
- **Latitude (纬度)**：连续型数值，从 32 到 42。集中在 33-37（靠近洛杉矶、旧金山）。
- **Housing Median Age (房屋年龄中位数)**：离散型数值，范围在 1 到 52。较多的房屋集中在较小年龄段，反映较多较新房屋。
- **Total Rooms (房间总数)**：连续型数值，但取值为正整数。少数房屋有大量房间。
- **Total Bedrooms (卧室总数)**：连续型正整数。类似房间总数，但数值较小，右偏。
- **Population (人口数)**：连续型正整数，从几十到数千。大部分区域人口相对较少。
- **Households (家庭数)**：连续型正整数，范围从几到数千。数值较小的家庭数较多。
- **Median Income (收入中位数)**：连续型浮点数，单位为 1 万美元。一般在 0 到 15 之间。收入较高地

区相对较少。

- **Median House Value (房价中位数)**: 连续型浮点数, 单位为美元。在 1 万到 50 万。高房价集中在较少区域。
- **Ocean Proximity (邻近海洋)**: 类别变量, 包含“<1H OCEAN”、“INLAND”、“NEAR OCEAN”等值。各类别频数不均, 靠近海洋的区域占少数。

1.3 数据预处理

1.3.1 独热编码

首先统计 ocean_proximity 的信息, 如下图所示:

```
ocean_proximity
<1H OCEAN    9136
INLAND       6551
NEAR OCEAN   2658
NEAR BAY     2290
ISLAND        5
Name: count, dtype: int64
(20433, 14)
```

使用如下代码将源数据转换为独热编码形式:

```
# 将 ocean_proximity 转换为独热编码
data_1 = pd.get_dummies(data, columns=['ocean_proximity'], prefix='ocean', dtype=int)
```

可以看到原来的分类型特征被拆分成了四个独立的特征, 分别由 0-1 来表示其是否为当前的特征:

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	...	ocean_<1H OCEAN	ocean_INLAND	ocean_ISLAND	ocean_NEAR BAY	ocean_NEAR OCEAN
0	-122.23	37.88	41.0	880.0	129.0	...	0	0	0	1	0
1	-122.22	37.86	21.0	7099.0	1106.0	...	0	0	0	1	0
2	-122.24	37.85	52.0	1467.0	190.0	...	0	0	0	1	0
3	-122.25	37.85	52.0	1274.0	235.0	...	0	0	0	1	0
4	-122.25	37.85	52.0	1627.0	280.0	...	0	0	0	1	0
...	...	...	...	...	...	...	...	...	...	...	...
20635	-121.09	39.48	25.0	1665.0	374.0	...	0	1	0	0	0
20636	-121.21	39.49	18.0	697.0	150.0	...	0	1	0	0	0

```
[20640 rows x 14 columns]
```

由于源数据中 total_bedrooms 中存在缺失值, 所以我们要将缺失数据行删除, 删除后结果为:

```
[20433 rows x 14 columns]
```

1.3.2 将因变量移至最右

先将 median_house_value 删掉并拼接到最右边:

```
tmp = data_1.drop("median_house_value", axis=1)
y = data_1["median_house_value"]
data_2 = pd.concat([tmp, y], axis=1)
print(data_2.shape)
print(data_2)
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	...	ocean_INLAND	ocean_ISLAND	ocean_NEAR BAY	ocean_NEAR OCEAN	median_house_value
0	-122.23	37.88	41.0	880.0	129.0	...	0	0	1	0	452600.0
1	-122.22	37.86	21.0	7099.0	1106.0	...	0	0	1	0	358500.0
2	-122.24	37.85	52.0	1467.0	190.0	...	0	0	1	0	352100.0
3	-122.25	37.85	52.0	1274.0	235.0	...	0	0	1	0	341300.0
4	-122.25	37.85	52.0	1627.0	280.0	...	0	0	1	0	342200.0
...	...	...	...	...	...	...	...	...	...	...	...
20635	-121.09	39.48	25.0	1665.0	374.0	...	1	0	0	0	78100.0
20636	-121.21	39.49	18.0	697.0	150.0	...	1	0	0	0	77100.0
20637	-121.22	39.43	17.0	2254.0	485.0	...	1	0	0	0	92300.0
20638	-121.32	39.43	18.0	1860.0	409.0	...	1	0	0	0	84700.0
20639	-121.24	39.37	16.0	2785.0	616.0	...	1	0	0	0	89400.0

1.3.3 划分样本

划分 70%为训练集, 30%为测试集, 划分后的结果为:

(1) 训练集(14303 rows x 14 columns)

训练集	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	...	ocean_INLAND	ocean_ISLAND	ocean_NEAR	BAY	ocean_NEAR	OCEAN	median_house_value
19566	-120.96	37.61	23.0	3497.0	887.0	...	1	0	0	0	0	0	93400.0
7292	-118.22	33.98	34.0	2225.0	753.0	...	0	0	0	0	0	0	128800.0
17618	-121.94	37.28	27.0	2859.0	464.0	...	0	0	0	0	0	0	327500.0
17518	-121.91	37.34	35.0	2189.0	607.0	...	0	0	0	0	0	0	240900.0
5172	-118.28	33.95	41.0	835.0	208.0	...	0	0	0	0	0	0	86200.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...
11397	-117.97	33.72	24.0	2991.0	500.0	...	0	0	0	0	0	0	273400.0
12081	-117.54	33.76	5.0	5846.0	1035.0	...	0	0	0	0	0	0	160800.0
5447	-118.42	34.01	42.0	1594.0	369.0	...	0	0	0	0	0	0	335400.0
866	-122.04	37.57	12.0	5719.0	1064.0	...	0	0	0	1	0	0	231200.0
15948	-122.43	37.73	52.0	3602.0	738.0	...	0	0	0	1	0	0	251800.0

(2) 测试集(6130 rows x 14 columns)

测试集	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	...	ocean_INLAND	ocean_ISLAND	ocean_NEAR	BAY	ocean_NEAR	OCEAN	median_house_value
14416	-117.24	32.79	20.0	961.0	278.0	...	0	0	0	0	1	0	245800.0
16383	-121.29	38.01	2.0	6403.0	1116.0	...	1	0	0	0	0	0	137900.0
7731	-118.14	33.92	31.0	3731.0	853.0	...	0	0	0	0	0	0	218200.0
1410	-122.07	37.94	30.0	1260.0	276.0	...	0	0	0	1	0	0	220800.0
1335	-121.89	37.99	4.0	2171.0	597.0	...	1	0	0	0	0	0	170500.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...
16764	-122.49	37.70	36.0	1946.0	340.0	...	0	0	0	0	1	0	287700.0
5762	-118.29	34.16	31.0	1262.0	338.0	...	0	0	0	0	0	0	241900.0
12862	-121.39	38.69	30.0	2897.0	506.0	...	1	0	0	0	0	0	88400.0
18814	-121.45	41.04	33.0	2029.0	378.0	...	1	0	0	0	0	0	77500.0
12852	-121.37	38.68	35.0	1620.0	276.0	...	1	0	0	0	0	0	72900.0

1.3.4 整合数据为 x-y 形式

将表格前 n-1 列赋给 x，最后一列赋给 y，所以先求表格总数，再转换 Xy 并输出：

```
# 将训练集中的数据取出
n = train_data.shape[1] # 求出表格的总列数 n
# 将 median_house_value 单独分出给 y，剩余数据给 X
X = train_data.iloc[:, 0:n - 1]
y = train_data.iloc[:, n - 1:n]
X_test = test_data.iloc[:, 0:n - 1]
y_test = test_data.iloc[:, n - 1:n]

print("X 矩阵为")
print(X)
print("\ny 矩阵为")
print(y)
```

X 矩阵：

X 矩阵为	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population	...	median_income	ocean<1H	OCEAN	ocean_INLAND	ocean_ISLAND	ocean_NEAR	BAY	ocean_NEAR	OCEAN
19566	-120.96	37.61	23.0	3497.0	887.0	2467.0	...	1.9444	0	0	1	0	0	0	0	0
7292	-118.22	33.98	34.0	2225.0	753.0	2980.0	...	1.6685	1	0	0	0	0	0	0	0
17618	-121.94	37.28	27.0	2859.0	464.0	1144.0	...	5.0822	1	0	0	0	0	0	0	0
17518	-121.91	37.34	35.0	2189.0	607.0	1193.0	...	2.8042	1	0	0	0	0	0	0	0
5172	-118.28	33.95	41.0	835.0	208.0	707.0	...	1.4103	1	0	0	0	0	0	0	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
11397	-117.97	33.72	24.0	2991.0	500.0	1437.0	...	5.4286	1	0	0	0	0	0	0	0
12081	-117.54	33.76	5.0	5846.0	1035.0	3258.0	...	4.7965	1	0	0	0	0	0	0	0
5447	-118.42	34.01	42.0	1594.0	369.0	952.0	...	3.0990	1	0	0	0	0	0	0	0
866	-122.04	37.57	12.0	5719.0	1064.0	3436.0	...	5.2879	0	0	0	0	0	1	0	0
15948	-122.43	37.73	52.0	3602.0	738.0	2270.0	...	3.8934	0	0	0	0	0	1	0	0

[14303 rows x 13 columns]

Y 矩阵：

```

y 矩阵为
      median_house_value
19566      93400.0
7292      128800.0
17618      327500.0
17518      240900.0
5172       86200.0
...
11397      273400.0
12081      160800.0
5447       335400.0
866        231200.0
15948      251800.0
> [14303 rows x 1 columns]

```

1.3.5 矩阵类型转换

```

# 将 X, y 分到的数据值转为矩阵类型，便于后续的计算
X = np.matrix(X.values)
y = np.matrix(y.values)
X_test = np.matrix(X_test.values)
y_test = np.matrix(y_test.values)

print("X 矩阵为")
print(X)
print("\ny 矩阵为")
print(y)
print("X 的形状为{}".format(X.shape))

```

X 矩阵:

```

X 矩阵为
[[-120.96  37.61  23.   ...  0.    0.    0. ]
 [-118.22  33.98  34.   ...  0.    0.    0. ]
 [-121.94  37.28  27.   ...  0.    0.    0. ]
 ...
 [-118.42  34.01  42.   ...  0.    0.    0. ]
 [-122.04  37.57  12.   ...  0.    1.    0. ]
 [-122.43  37.73  52.   ...  0.    1.    0. ]]

```

Y 矩阵:

```

y 矩阵为
[[ 93400.]
 [128800.]
 [327500.]
 ...
 [335400.]
 [231200.]
 [251800.]]

```

形状为:

```

X      为(14303, 13)
y      为(14303, 1)

```

1.3.6 特征缩放

将 x 特征进行缩放，并且为 X 和 X_test 引入一列偏执项:

```
def norm(X):
    sigma = np.std(X, axis=0) # axis=0 计算每一列的标准差;=1 计算行的
    miu = np.mean(X, axis=0)
    X = (X - miu) / sigma
    return X

X = norm(X)
X_test = norm(X_test)

X = np.c_[np.ones(y.size), X] # 矩阵合并, 第一列加1
X_test = np.c_[np.ones(y_test.size), X_test]
```

X 添加后为:

```
X 为
[[ 1. -0.68684567  0.92079826 ... -0.01182583 -0.35702009
  -0.38351632]
 [ 1.  0.68114892 -0.77889909 ... -0.01182583 -0.35702009
  -0.38351632]
 [ 1. -1.1761284  0.76628032 ... -0.01182583 -0.35702009
  -0.38351632]
 ...
 [ 1.  0.5812953 -0.76485201 ... -0.01182583 -0.35702009
  -0.38351632]
 [ 1. -1.22605521  0.90206881 ... -0.01182583  2.80096282
  -0.38351632]
 [ 1. -1.42076977  0.9769866  ... -0.01182583  2.80096282
  -0.38351632]]
```

X_test 添加后为:

```
X_test 为
[[ 1.  1.1468547 -1.31893689 ... -0.02212772 -0.34529097
  2.59237401]
 [ 1. -0.87353685  1.12310835 ... -0.02212772 -0.34529097
  -0.38574681]
 [ 1.  0.6978788 -0.79029492 ... -0.02212772 -0.34529097
  -0.38574681]
 ...
 [ 1. -0.92342307  1.44122919 ... -0.02212772 -0.34529097
  -0.38574681]
 [ 1. -0.95335479  2.54061737 ... -0.02212772 -0.34529097
  -0.38574681]
 [ 1. -0.91344582  1.43655094 ... -0.02212772 -0.34529097
  -0.38574681]]
```

各自的形状从原来的 13 变为 14, 符合我们的预期:

```
X      为(14303, 14)
X_test 为(6130, 14)
```

2. 线性回归模型的梯度下降参数优化

2.1 模型参数

- **θ :** 14×1 的列向量, 是模型的参数向量。用于存储线性回归模型的权重 (包括偏置项和各特征的权重)。在梯度下降过程中, 会不断更新, 使模型的预测值逐渐逼近真实房价。每个 θ_i 都与一个房价特征 (房屋面积、房间数、位置等) 相对应, 表示该特征对房价的影响程度。
- **steps:** 控制梯度下降的迭代次数。在每次迭代中, θ 都会更新一次, 逐渐减少误差。过少的迭代可能导致未收敛, 过多则增加计算时间。
- **α :** 学习率, 控制梯度下降每次更新 θ 的步长。合适的学习率让 θ 更新得既快又稳, 不至于错过或震荡于最优解。
- **J (代价向量):** 记录每次迭代的代价 (损失) 值, 表示模型预测与真实房价之间的差距。监控模型在训

练过程中是否收敛以及收敛速度。

这里我们事先定义好损失函数和梯度下降函数：

```
# 定义代价函数
def cost(theta, X=X, y=y, n=y.size):
    h_x = X @ theta
    inner = np.sum(np.power(h_x - y, 2))
    return inner / (2 * n)

# 梯度下降函数
def gradient_descend(theta, alpha):
    for i in range(num_iteration):
        J[i] = cost(theta, X) # 将每次迭代的代价函数值计入
        theta = theta - (alpha / y.size) * (X.T @ (X @ theta - y))
    return theta
```

而后我们设定好初始的 θ 、steps、 α ，其中迭代次数为 20000，学习率为 0.01：

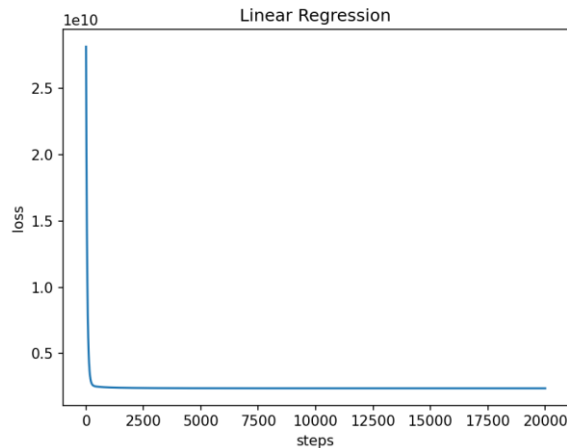
```
# 初始化 theta 的值 (theta 用于梯度下降中)
theta = np.matrix(np.zeros((X.shape[1], 1)))
theta_2 = theta

num_iteration = 20000 # 初始化迭代次数
alpha = 0.01 # 初始化学习速率
```

运行程序结果如下：

```
学习 为0.01
为[[207283.98035376]
 [-52444.90928561]
 [-53228.35855504]
 [ 13899.45857295]
 [-13922.45194753]
 [ 43292.13773274]
 [-40597.2881008 ]
 [ 15822.61629881]
 [ 74721.78132984]
 [ 6469.81408922]
 [-12584.82519642]
 [ 2443.37694102]
 [ 2396.17813651]
 [ 5480.88410642]]
```

损失随着 steps 的增加而逐步降低，结果如下：



2.2 求解 R2

我们定义 R2 的求解方程：

```
def R_2(X_test, y_test, theta):
    y_pred = X_test * theta
    mu = np.mean(y_test, axis=0)
    SSE = np.sum(np.power(y_test - y_pred, 2))
    SSR = np.sum(np.power(y_pred - mu, 2))
    SST = SSR + SSE
    r_2 = 1 - SSE / SST
    return r_2
```

而后我们求解的 R2 结果如下，表明模型在测试集上的解释能力为 64.84%。即模型能够解释测试数据中 64.84% 的方差，其余 35.16% 的方差仍然未被模型捕捉。由于 R^2 值越接近 1，模型的预测能力越好。 $R^2 = 0.6484$ 属于中等偏上的拟合水平，表明模型有一定的预测能力，但仍有提升空间。

测试集	R2:
0.6484183067329626	

3. 线性回归模型的闭合形式参数求解

我们定义正规方程，直接求取线性方程组的解：

```
def NormalEquation(theta_2):
    theta_2 = np.linalg.inv(X.T @ X) @ X.T @ y # 求矩阵的逆
    return theta_2
```

求取的 θ 如下：

```
为[[207283.98035376]
 [-40294.97048095]
 [-40471.87453306]
 [ 14223.13727057]
 [-16180.13634171]
 [ 48071.44902976]
 [-39207.48143814]
 [ 12067.71901347]
 [ 75540.0999077 ]
 [ 17285.45889461]
 [-6750.65824967]
 [ 2730.99840899]
 [ 10626.72315349]
 [ 11609.81617421]]
```

求取的 R2 结果如下：

测试集 R2:
0.6485794062545698

可以看到 R^2 相比于梯度下降的方法的要略微升高，这表明两种方法在模型效果上较为接近，但正规方程无需迭代计算，实现上更为直接。其性能略优的原因在于，正规方程能够一步求解出损失函数的全局最优参数，不受学习率选择和收敛速度的影响；而梯度下降可能因迭代次数不足或超参数设置不够理想，未能完全收敛至最优解。因此，从理论上讲，正规方程可以求得更为精确的参数结果。

4. 在测试集上计算所训练模型的 R^2 指标

由上述可以得出，在测试集上分别使用梯度下降和正规方程所训练的模型对应的 R^2 结果如下：

	梯度下降	正规方程
R^2	0.6484	0.6485

可以发现正规方程的方法对应的 R^2 要更具优势，原因可能如下：

- (1) 梯度下降法可能因学习率或迭代次数设置不当而未能达到全局最优。学习率过大容易导致参数震荡，过小则收敛缓慢；同时，梯度下降也可能陷入局部最优，使得最终 R^2 略低于正规方程的结果。
- (2) 正规方程通过解析方式直接求解全局最优参数，无需设置学习率、迭代次数等超参数。对于中小规模数据集，具有计算效率高、求解精度好的优点。
- (3) 两种方法得到的 R^2 值相差不大，说明模型整体训练效果较为理想。

5. 讨论实验结果，分析特征与预测值的正负相关性

计算各个特征与目标变量之间的相关系数，结果如下：

```
ocean_INLAND      -0.484787
latitude           -0.144638
longitude          -0.045398
population         -0.025300
ocean_ISLAND       0.023525
total_bedrooms     0.049686
households         0.064894
housing_median_age 0.106432
total_rooms        0.133294
ocean_NEAR OCEAN   0.140378
ocean_NEAR BAY     0.160526
ocean_<1H OCEAN    0.257614
median_income      0.688355
median_house_value 1.000000
Name: median_house_value, dtype: float64
```

结合相关系数我们可以分析得到：

(1) 收入是影响房价的最核心因素

与房价正相关最强的特征是 `median_income`（中位收入），相关系数达到 0.688，说明收入水平与房价之间存在较强的正相关关系。收入越高的区域，房价也越高，这与现实经济规律高度吻合。

(2) 地理位置对房价具有显著影响

与房价负相关最强的特征是 `latitude`（纬度），相关系数为 -0.144，表明随着纬度增加（即向北移动），房价呈下降趋势，反映出加利福尼亚州南部地区房价普遍高于北部地区。此外，`ocean_INLAND` 与房价呈较强负相关（-0.485），说明内陆地区的房价明显低于沿海区域，进一步印证了地段对房价的重要性。

(3) 人口数量与房价几乎无关，房龄影响较为有限

`population`（人口）的相关系数仅为 -0.025，接近于零，表明区域人口规模与房价之间几乎不存在线性关系。而 `housing_median_age`（房龄中位数）的相关系数为 0.106，呈现一定程度的正相关，说明较新的社区或房屋可能具有更高的价值，但影响程度相对较弱，不及收入水平和地理位置的作用。

四、实验结果

1. 训练所得的 θ 值（梯度下降与正规方程）

梯度下降：

```
学习 为0.01
为[[207283.98035376]
[-52444.90928561]
[-53228.35855504]
[ 13899.45857295]
[-13922.45194753]
[ 43292.13773274]
[-40597.2881008 ]
[ 15822.61629881]
[ 74721.78132984]
[ 6469.81408922]
[-12584.82519642]
[ 2443.37694102]
[ 2396.17813651]
[ 5480.88410642]]
```

正规方程：

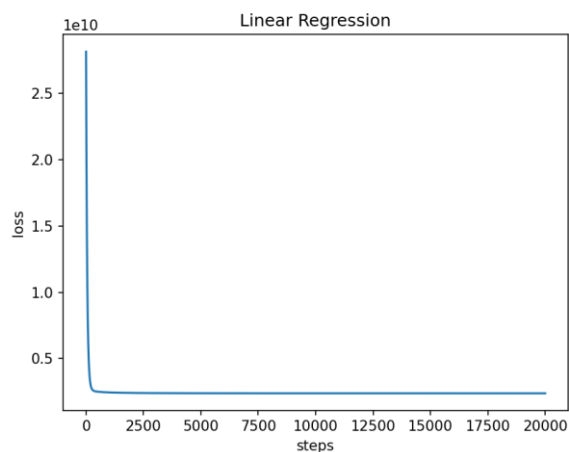
```
为[[207283.98035376]
[-40294.97048095]
[-40471.87453306]
[ 14223.13727057]
[-16180.13634171]
[ 48071.44902976]
[-39207.48143814]
[ 12067.71901347]
[ 75540.0999077 ]
[ 17285.45889461]
[-6750.65824967]
[ 2730.99840899]
[ 10626.72315349]
[ 11609.81617421]]
```

2. 两种方法所得的 R2

	梯度下降	正规方程
R2	0.6484	0.6485

分析可见实验步骤的第三部分

3. 损失函数迭代结果



4. 分析各特征与目标变量的正负相关系数

特征	相关系数	相关性强度	解释
median_income	0.688	强正相关	收入是影响房价的最重要因素，收入越高，房价越高
ocean_<1H OCEAN	0.258	中等正相关	距离海洋不到 1 小时的区域，房价相对较高

ocean_NEAR BAY	0.161	弱正相关	靠近海湾的区域，房价有一定优势
ocean_NEAR OCEAN	0.140	弱正相关	靠近海洋的区域，房价略高
total_rooms	0.133	弱正相关	房间总数越多，房价略高（可能与房屋面积相关）
housing_median_age	0.106	弱正相关	房龄越大（社区越成熟），房价略高
households	0.065	极弱正相关	家庭数与房价几乎无关
total_bedrooms	0.050	极弱正相关	卧室数量对房价影响很小
ocean_ISLAND	0.024	极弱正相关	岛屿地区的样本极少（仅 5 个），参考价值有限
population	-0.025	极弱负相关	人口数量与房价几乎无线性关系
longitude	-0.045	极弱负相关	经度越大（位置越东），房价略有下降
latitude	-0.145	弱负相关	纬度越高（位置越北），房价越低
ocean_INLAND	-0.485	中等负相关	内陆地区房价显著较低，是房价的主要负面因素

分析可见实验步骤 5 中的具体分析，这里略。

五、实验体会

通过本次基于 California Housing Prices 数据集的线性回归实验，我系统地掌握了机器学习中线性回归模型的完整实现流程，包括数据探索、预处理、模型训练、参数求解和结果评估等关键环节。

在数据处理阶段，我深刻体会到数据预处理对模型性能的重要性。首先，通过将类别型特征 `ocean\_proximity` 转换为 one-hot 编码，使非数值特征能够被模型有效利用。其次，处理 `total\_bedrooms` 中的缺失值，确保了数据质量。此外，对数值型特征进行标准化处理，有效消除了不同量纲对梯度下降收敛速度的影响，同时为特征矩阵添加偏置项，使得截距项能够与权重参数统一更新。这些预处理操作让我认识到，真实数据往往存在各种不规范性，如何合理处理数据是机器学习实践中至关重要的一步。

在模型训练方面，我独立实现了梯度下降和正规方程两种参数求解方法。梯度下降通过迭代逐步逼近最优解，让我直观感受到学习率、迭代次数等超参数对收敛过程的影响；而正规方程则通过解析形式一步求解全局最优，避免了超参数调优的繁琐。两种方法的 R^2 均在 0.648 左右，表现相近，验证了实现的正确性。通过对比，我理解了梯度下降适用于大规模数据，而正规方程在中小规模数据上更具计算优势。

在结果分析中，我计算了各特征与房价的相关系数，发现 `median\_income` 与房价呈强正相关（0.688），`ocean\_INLAND` 呈强负相关（-0.485），这与现实情况高度吻合，说明模型具备一定的可解释性。同时，`population` 的相关系数接近于零，表明人口规模对房价几乎没有线性影响。这些分析让我认识到，机器学习不仅关注预测精度，也要注重模型的解释能力。

本次实验让我从底层实现角度理解了线性回归的数学原理，提升了编程实践和问题分析能力。更重要的是，我学会了如何在数据预处理、模型选择、参数调优和结果分析等环节中做出合理决策，为后续学习更复杂的机器学习模型奠定了坚实基础。

教师评语：

指导教师：

日 期：